

ALL PROGRAMMABLE

ANY MEDIA

5G

4K/8K

ANY STANDARD

ANY MACHINE

ANY NETWORK

5G Wireless • Embedded Vision • Industrial IoT • Cloud Computing



Building customized Overlay



Agenda

- Overlay design methodology
- Building custom Overlay
- Labs
 - Developing a single IP
 - Writing a python-driver
 - Creating a custom overlay

Overlay design methodology

Overlay design considerations

➤ Programmable Logic design

- Zynq PS configuration must match the settings defined for PYNQ
- Zynq architecture & interfaces between PS-PL
- PL design – reusable IP
- Overlay programmability

➤ New IOP peripheral

- Software design of IOP driver for custom peripheral

➤ Python interface

- May be pure Python wrapper
- CFFI to wrap C driver with thin Python layer

Pynq-Z2 base configuration

➤ PYNQ image used to boot board

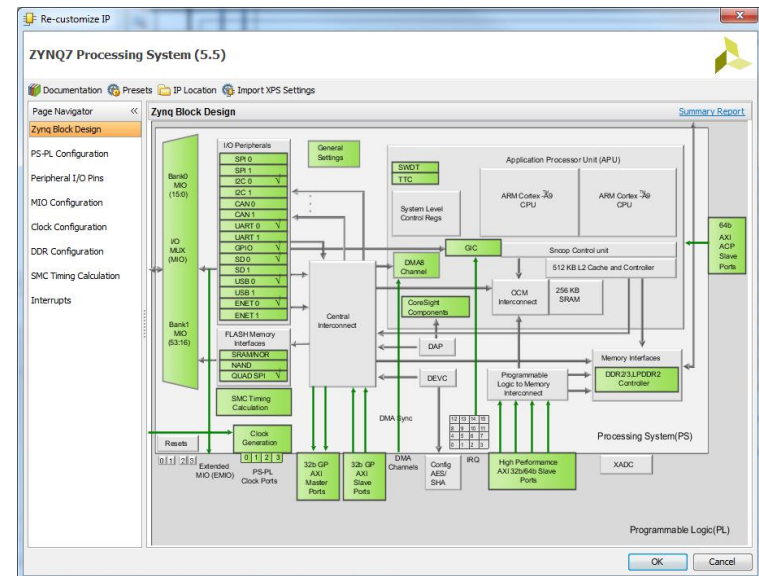
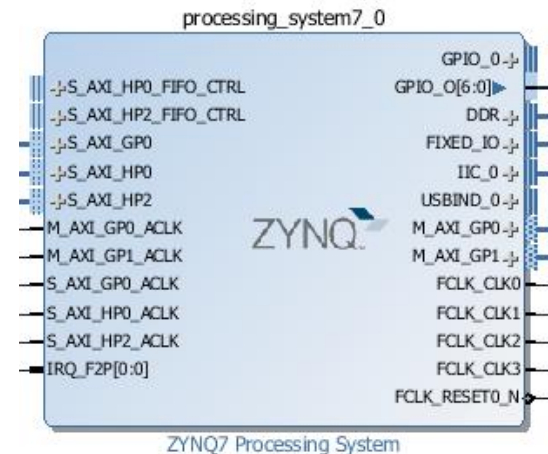
- PS is configured during boot
- Default overlay loaded

➤ Downloading new overlays

- Only the PL design from Vivado (bitstream) will be downloaded

➤ Designing new overlays

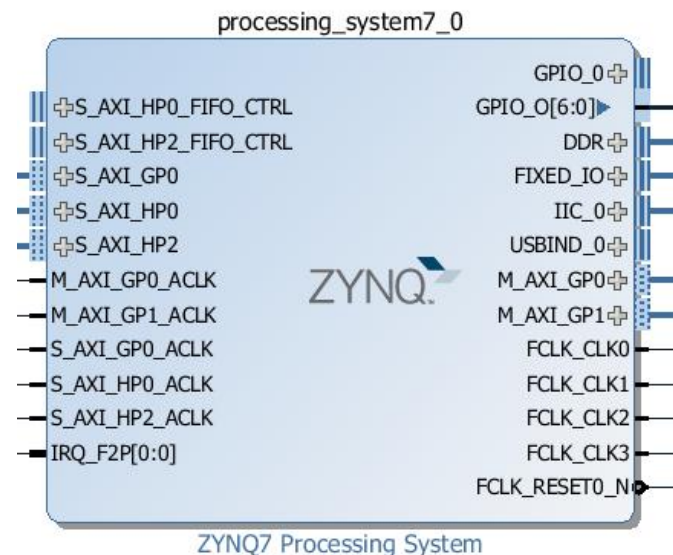
- PL depends on PS e.g. clocks from PS
- Vivado PS settings must match PYNQ image



Zynq PS settings

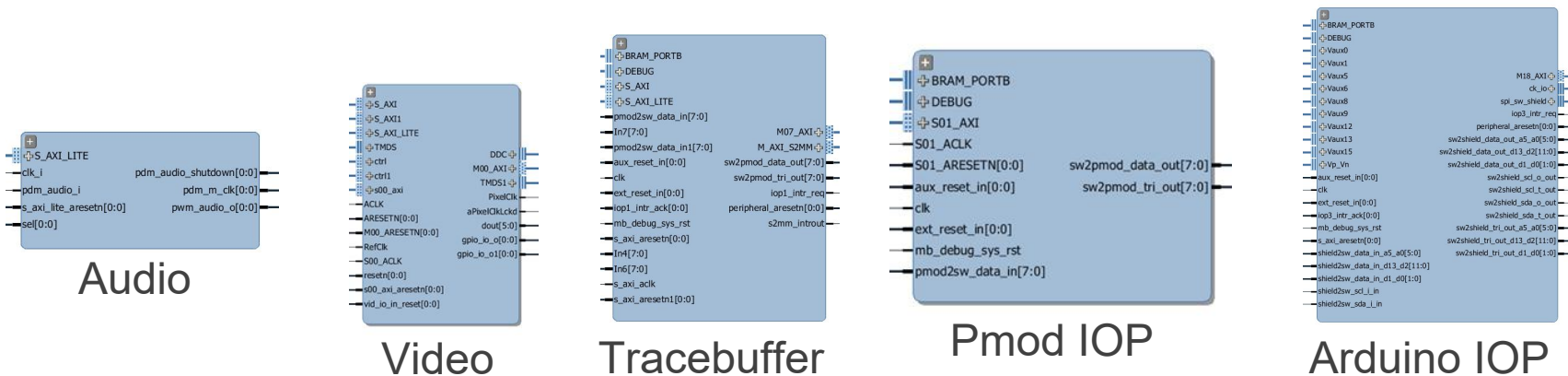
Pynq-Z2 PS Settings

- GP0, GP1 enabled
 - IOPs, Video control, GPIO, Audio, Interrupts
 - Tracebuffer
- HP0, HP2
 - Video
 - Tracebuffer
- Peripherals
 - SD, Ethernet, GPIO, IIC (Video), USB, Uart
- Clocks
 - 100 MHz, 142 MHz, 200 MHz
 - Software Programmable



PL Design

- Standard FPGA/Zynq design
- IP version and driver depends on Vivado version
 - PYNQ drivers also depend on Vivado
- Current version of Vivado: 2016.1
 - Other versions may work, but are unsupported (driver differences)
- PYNQ reusable blocks
 - Audio, Video, Tracebuffer, IOPs



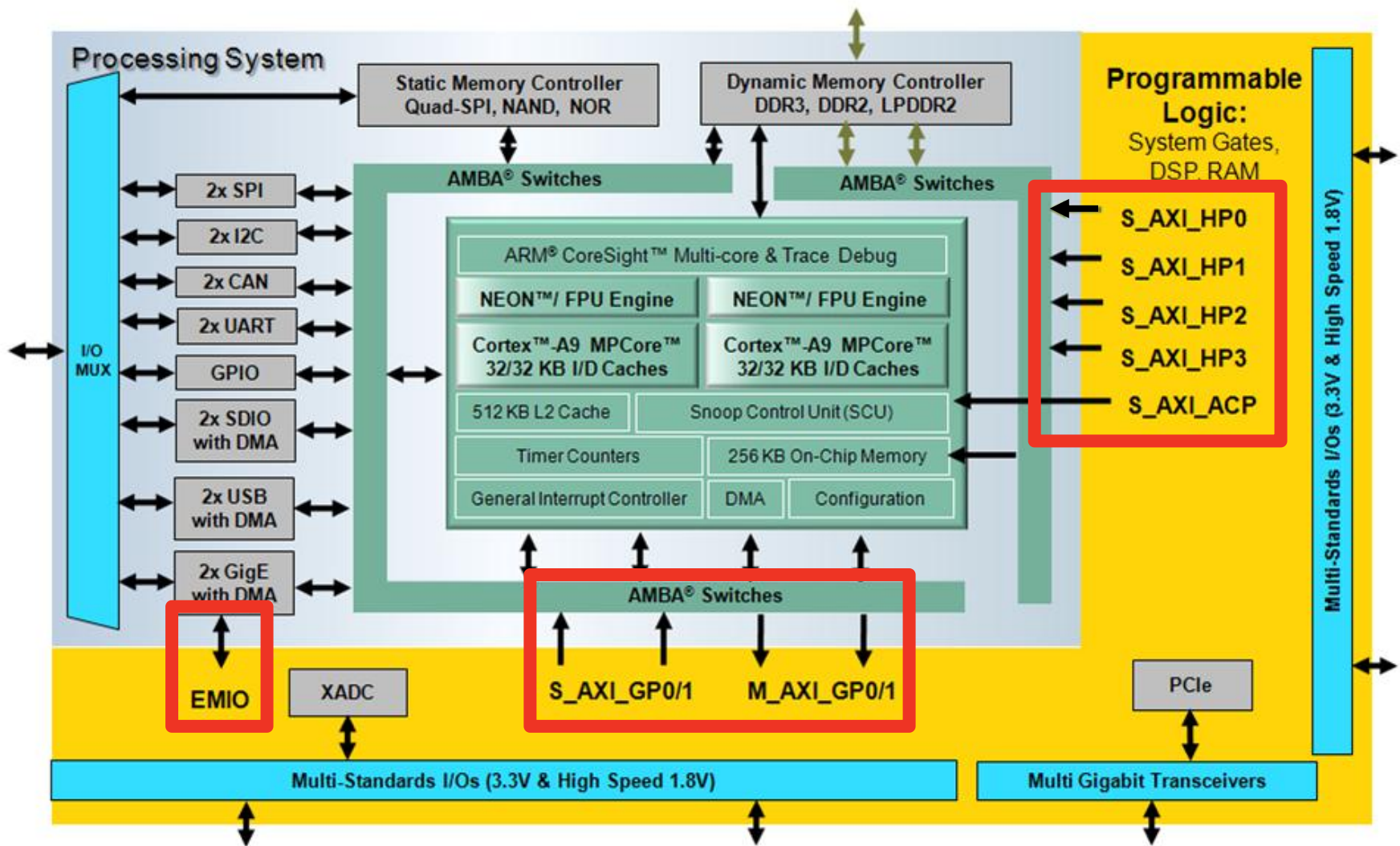
Overlay Programmability

- End-user programmability and flexibility valued over logic utilisation
 - Create Programmable IP and overlays that can be reused for many applications and domains
- IOPs support many peripherals in the same design
- E.g.
 - BNN Design
 - Weights can be “hard coded” into bitstream for slightly higher performance/resource utilisation
 - However, each network would need to be built with Xilinx tools
 - BNN Overlay
 - Weights can be programmed into BRAM at runtime for flexibility
 - Many networks can run on the same overlay

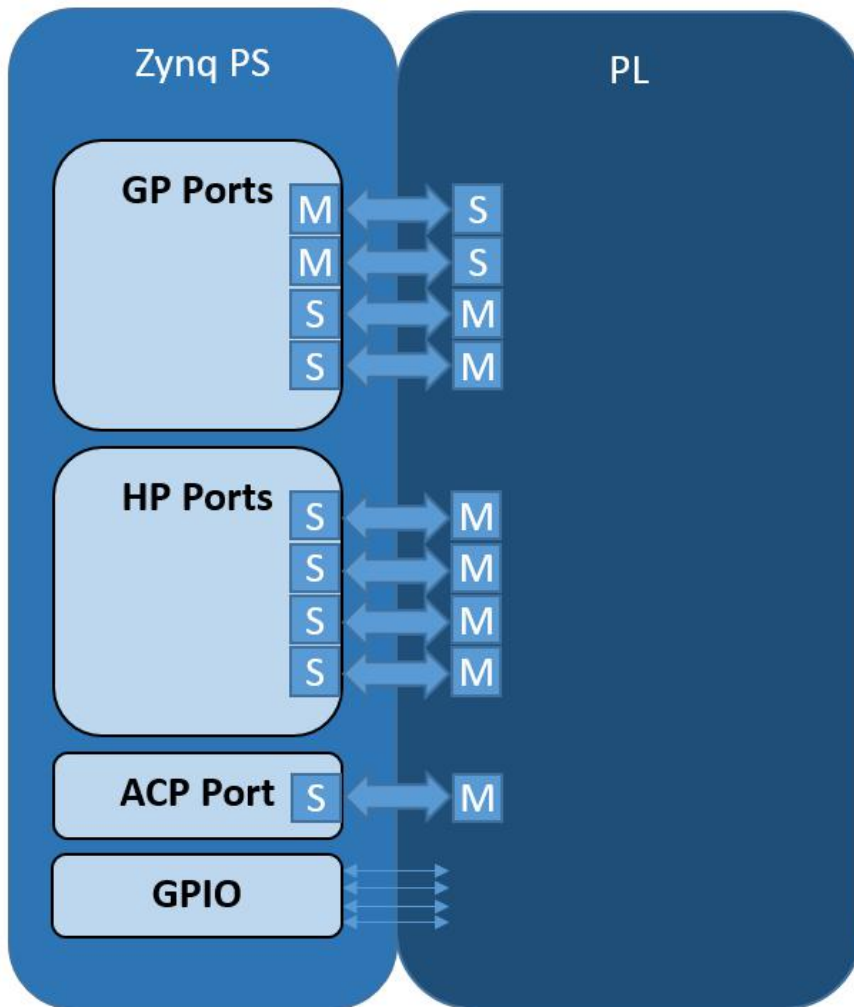
Overlay TCL

- Bitstream + TCL must be provided to load an overlay
 - [\\pynq\xilinx\pynq\bitstream](#)
- TCL from Vivado block diagram is used to provide information about the system
 - ip_dict: addresses of IP block
 - gpio_dict: used internally to connect IOP resets/interrupts
 - EMIO GPIO pins, interrupt pins
- Used internally in PYNQ
 - E.g. Instantiating IOPs
- Must generate TCL from Vivado IP Integrator Block Diagram
 - “Standardizes” TCL scripts
 - File > Export > Export Block Design
 - write_bd_tcl command

Zynq PS-PL interfaces

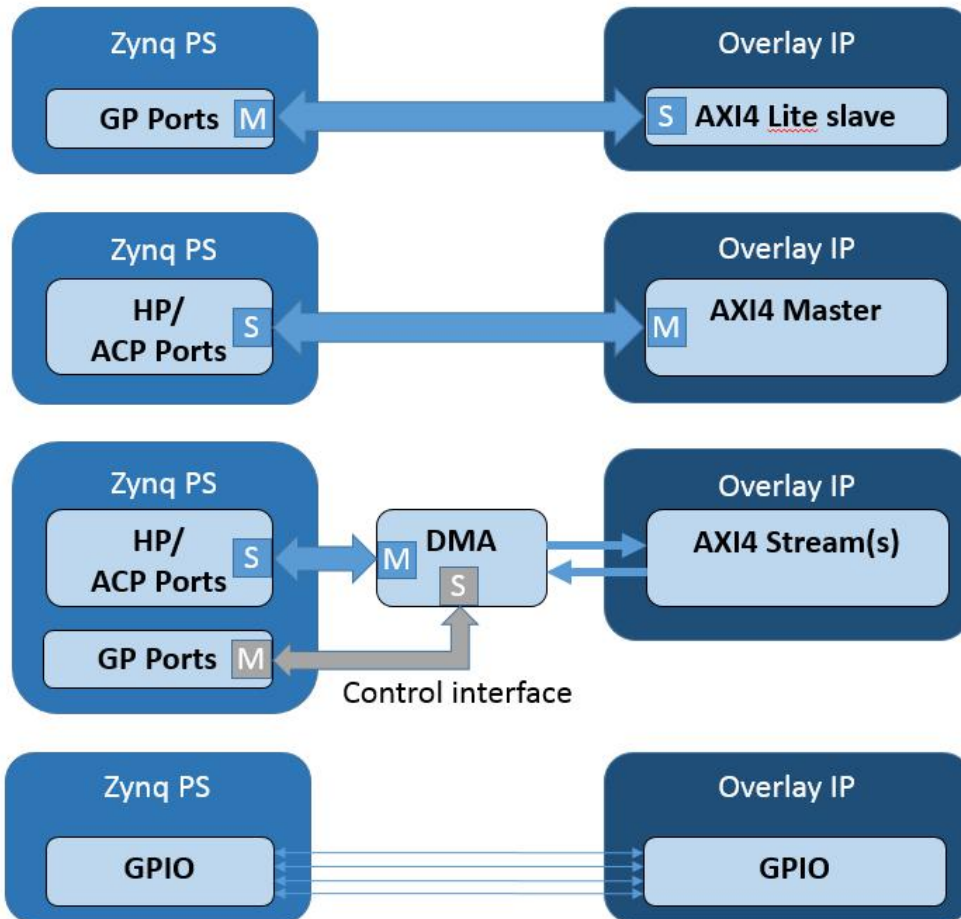


PS-PL interfaces types



- AXI General Purpose ports
 - 2x Master (32-bit)
 - 2x Slave (32-bit)
- AXI High Performance
 - 4x Slave (64-bit)
 - 1K FIFOs
- ACP
 - 1x Slave (64-bit)
 - Cache access
- GPIO (EMIO)
 - Wires (64)

IP interfaces



➤ AXI General Purpose

- Typically AXI Lite slaves
- Low performance

➤ AXI HP/ACP

- AXI4 Master
- Higher performance

➤ DMA

- HP/ACP ports
 - AXI Stream

➤ GPIO

- Control type data

Building custom Overlay

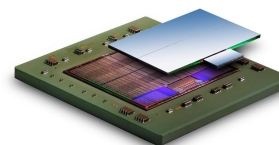
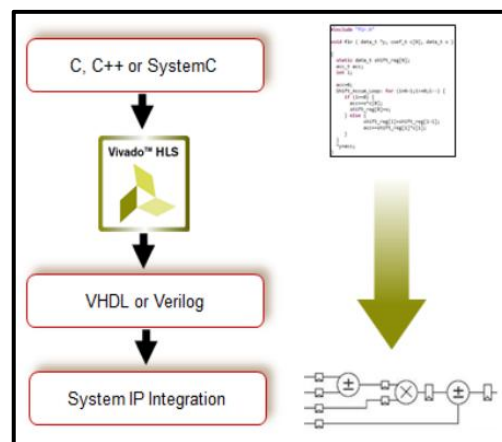
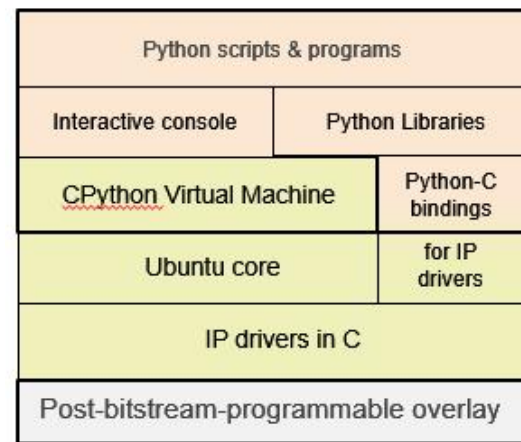
Overview

➤ Design goals

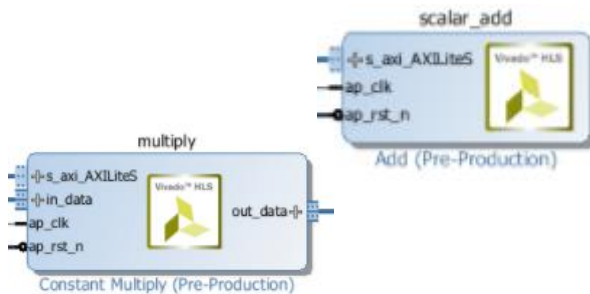
- Allow overlay users to find out what is inside an overlay in a consistent manner
- Provide a simple way for developers of new hardware designs to test new IP
- Facilitate reuse of IP between Overlays

➤ How to

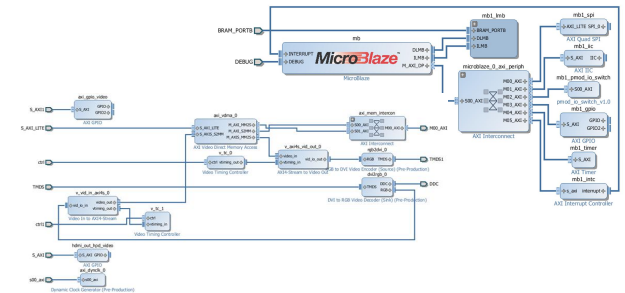
- Walking through the process of interacting with a new IP
- Developing a python driver
- Finally building a more complex system from multiple IP blocks



Steps of building custom Overlays



Step 1:
Developing a single IP with specific function.



Step 2:
Create an FPGA design for a class of related applications

```
class ConstantMultiplyDriver(DefaultIP):
    def __init__(self, description):
        super().__init__(description=description)

        bindto = ['Xilinx:hls:mult_constant:1.0']

        @property
        def constant(self):
            return self.read(0x10)

        @constant.setter
        def constant(self, value):
            self.write(0x10, value)
```

Step 3:
Writing a Python-driver for the FPGA design

```
6c78 3963 7367 3232 3500 6300 0b32
332f 3039 2f33 3000 6400 0931 323a
3a31 3900 6500 0532 7c3f ffff ffff
ffff ffff ffff ffff ffaa 9955 6630
0720 0031 a103 8031 413d 0831 6109
c204 0010 9330 e100 cf30 c100 8120
0020 0020 0020 0020 0020 0020 0020
0020 0020 0020 0020 0020 0020 0020
813c c831 8108 8134 2100 0032 0100
01
00
```

```
import pynq.lib.dma

overlay = Overlay('/home/xilinx/tutorial_2.bit')

dma = overlay.const_multiply.multiply_dma
multiply = overlay.const_multiply.multiply
```

Step 4:
Import the bitstream and the library in your Python scripts and program

Step 0: Checking prerequisite

➤ Find folder *overlay_tutorial*

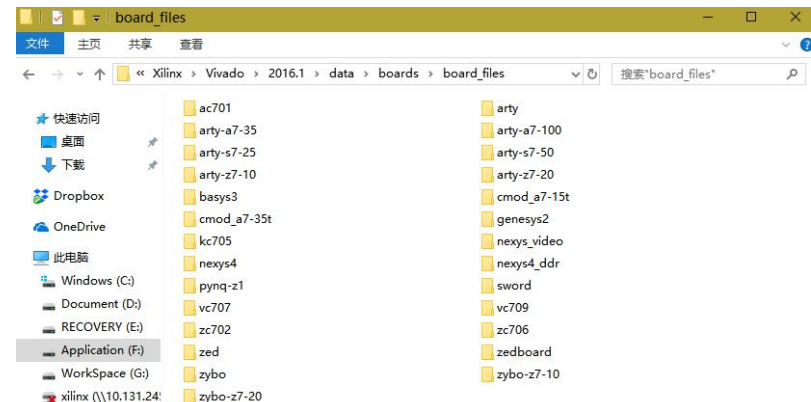
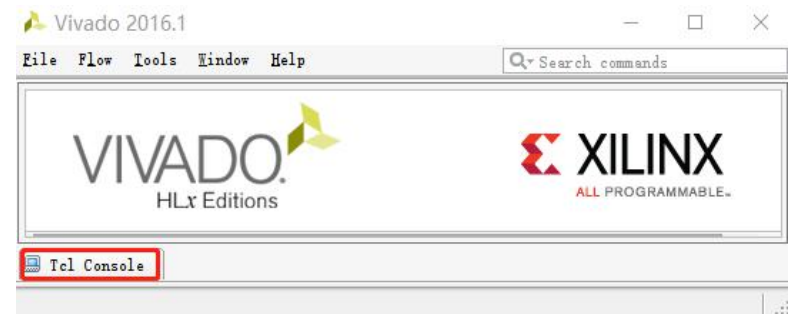
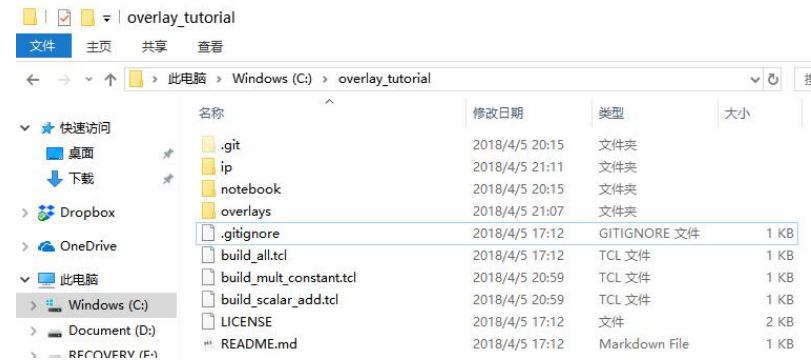
- You can also download from
- https://github.com/sonnyhcl/overlay_tutorial/archive/master.zip

➤ Vivado suite pre-installed

- Version 2016.1
- Open it and click into *Tcl Console*
- Type *cd path/to/overlay_tutorial*
 - E.g. *cd C:/overlay_tutorial*

➤ Make sure Digilent board files exists

- You can download from
- <https://github.com/Digilent/vivado-boards/archive/master.zip>



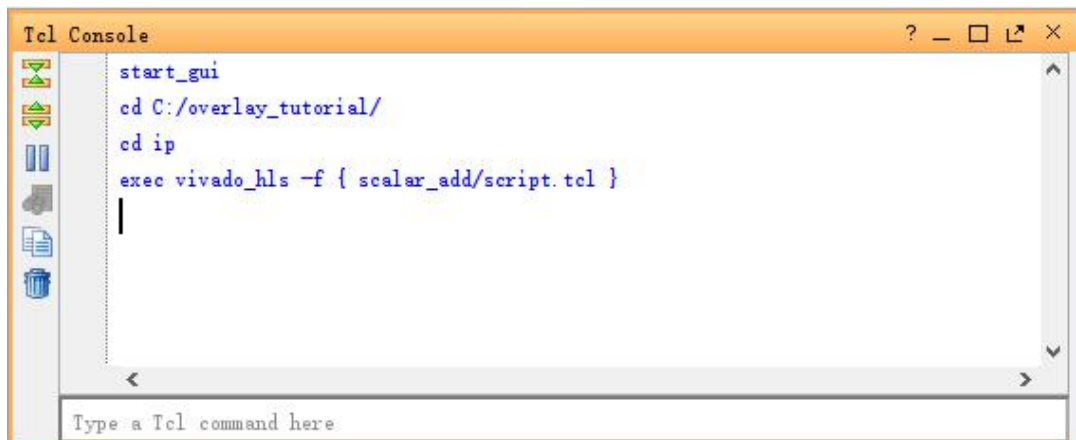
Step 1: Develop a single IP using HLS

➤ Writing C/C++ code using HLS

– You can find src code in [C:\overlay_tutorial\ip\scalar_add\add.cpp](#)

```
void add(int a, int b, int& c) {  
#pragma HLS INTERFACE ap_ctrl_none port=return  
#pragma HLS INTERFACE s_axilite port=a  
#pragma HLS INTERFACE s_axilite port=b  
#pragma HLS INTERFACE s_axilite port=c  
  
    c = a + b;  
}
```

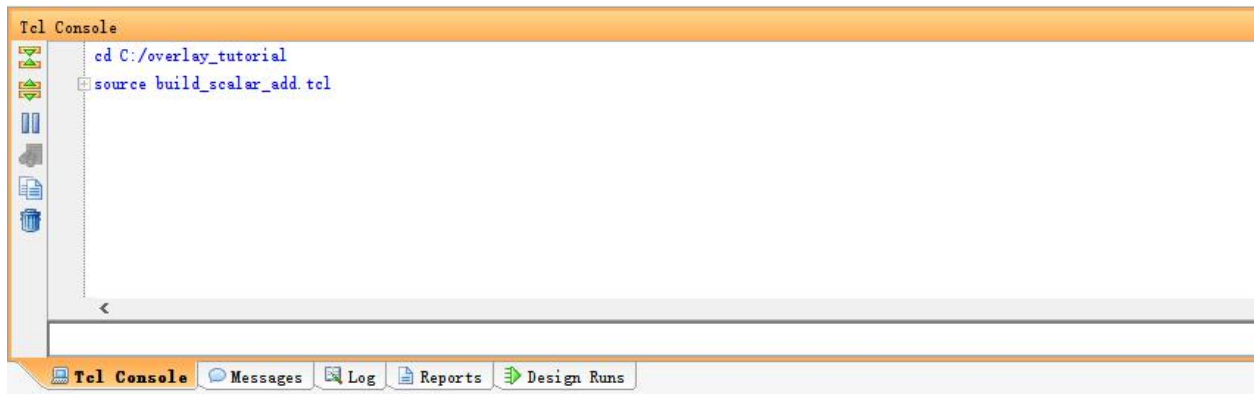
➤ Generate IP using tcl script



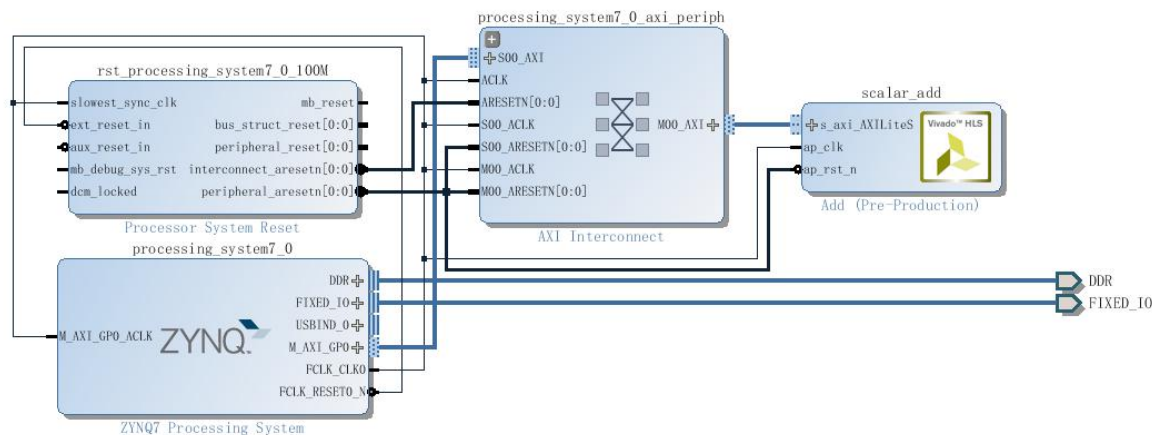
Step 2: Create FPGA design using Vivado

➤ Creating an FPGA design

- Generate project using tcl `C:\overlay_tutorial\build_scalar_add.tcl`



➤ Open Block Design



Step 3: Write Python driver

➤ Test the functionality of FPGA design

```
from pynq import Overlay

overlay = Overlay('../overlays/tutorial_1.bit')
add_ip = overlay.scalar_add
add_ip.write(0x10, 4)
add_ip.write(0x18, 5)
add_ip.read(0x20)
```

9

➤ Write the *AddDriver* for FPGA design

```
from pynq import DefaultIP

class AddDriver(DefaultIP):
    def __init__(self, description):
        super().__init__(description=description)

        bindto = ['xilinx.com:hls:add:1.0']

    def add(self, a, b):
        self.write(0x10, a)
        self.write(0x18, b)
        return self.read(0x20)
```

Step 3: Write Python driver

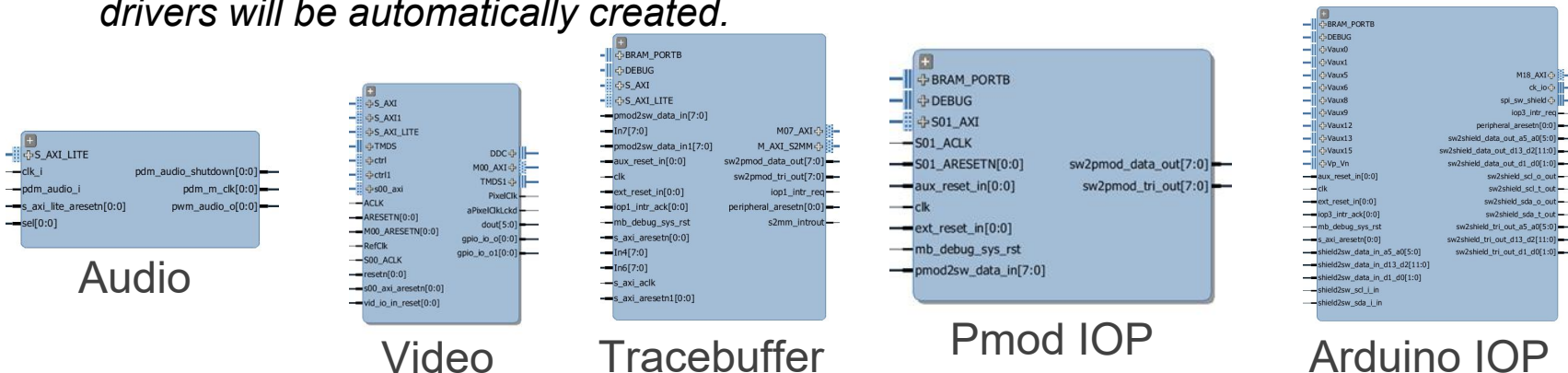
➤ Load *scalar_add*

```
overlay = Overlay('../overlays/tutorial_1.bit')  
  
overlay.scalar_add.add(15, 20)
```

35

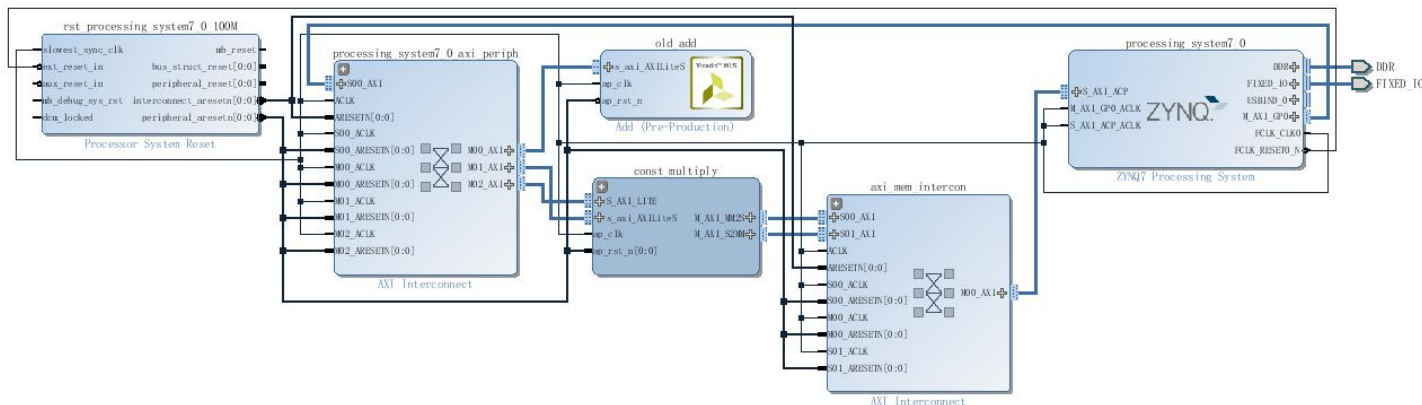
➤ Reusing IP in next Project - *mult_constant*

- Suppose we or someone else develops a new overlay and wants to reuse the existing IP.
- As long as they import the python file containing the driver class, the drivers will be automatically created.



Make your hands dirty

➤ Reusing IP in next Project - *mult_constant*



➤ Same work flow as *scala_add*

- Build *mult_constant* IP using HLS
- Create FPGA design using Vivado
- Write Python driver

➤ Open your Jupyter Notebook and have a try !!!

- <http://192.168.2.99:9090>
- Username: xilinx
- Password: xilinx

Step 4: Import as new Overlay

➤ Create drivers for *mult_constant*

```
from pyng import DefaultHierarchy

class StreamMultiplyDriver(DefaultHierarchy):
    def __init__(self, description):
        super().__init__(description)

    def stream_multiply(self, stream, constant):
        self.multiply.constant = constant
        with xlnk.cma_array(shape=(len(stream)), \
                             dtype=np.uint32) as in_buffer, \
             xlnk.cma_array(shape=(len(stream)), \
                             dtype=np.uint32) as out_buffer:
            for i, v, in enumerate(stream):
                in_buffer[i] = v
            self.multiply_dma.sendchannel.transfer(in_buffer)
            self.multiply_dma.recvchannel.transfer(out_buffer)
            self.multiply_dma.sendchannel.wait()
            self.multiply_dma.recvchannel.wait()
            result = out_buffer.copy()
        return result

    @staticmethod
    def check_hierarchy(description):
        if 'multiply_dma' in description['ip'] \
            and 'multiply' in description['ip']:
            return True
        return False
```

➤ Creating a custom overlay

```
class TestOverlay(Overlay):
    def __init__(self, bitfile, **kwargs):
        super().__init__(bitfile, **kwargs)

    def multiply(self, stream, constant):
        return self.const_multiply.stream_multiply(stream, constant)
```

➤ Call *mult_constant* Overlay

```
overlay = TestOverlay('/home/xilinx/tutorial_2.bit')
overlay.multiply([2,3,4,5,6], 4)
```

```
ContiguousArray([ 8, 12, 16, 20, 24], dtype=uint32)
```

Summary

- Overlay design methodology
- Build custom Overlay
- Labs
 - Developing a single IP
 - Creating a python-driver
 - Creating a custom overlay

Questions?